nxalien

A Post-Symbolic Agent-Context Exoskeleton for PSE
Hypercube-HDAG Specification and Implementation Handoff

Sebastian Klemm
Specification draft for implementation by Claude Code / Sonnet 4.6

May 2026

Abstract

`nxalien` is specified as a post-symbolic replacement for ordinary AI rule installers. It takes the useful surface idea of an IDE-agnostic context file generator and lifts it into a deterministic, auditable, Hypercube-HDAG-based extension layer for the Post-Symbolic Engine (PSE). Instead of merely writing a static `.rules` file and an append-only log, `nxalien` compiles project context, engineering rules, unknown decisions, evidence requirements, specialist triggers, and validation gates into a canonical *Agent Context Hypercube*. The cube embeds an executable HDAG, materializes context artifacts, emits PSE handoff candidates, and can be committed to PSE/IL only through existing fail-closed bridge semantics. The design goal is not to copy `nxhuman`, but to produce an actually PSE-native context exoskeleton: deterministic, replayable, QTIC-compatible, path-invariant, and suitable for grounding coding agents.

Contents

1 Executive Summary

1.1 Result of the `nxhuman` Codebase Review

The uploaded `nxhuman` repository is a compact Node.js CLI package. Its actual implementation consists of a single executable `index.js`, a `package.json`, README, changelog, publishing notes, and license. The package generates two project-local files:

- `.rules`: a universal AI IDE context file containing engineering principles, workflow preferences, specialist triggers, quality standards, project metadata, and unknown-decision slots.
- `.nxlogs`: an append-only decision log initialized with project name, timestamp, and instructions for later manual/appended decision tracking.

It also optionally creates a `.cursorrules` symlink for Cursor compatibility. Its core philosophy is clear: evidence over assumptions, code over documentation, efficiency over verbosity. Technically, however, it has no post-symbolic substrate, no graph runtime, no deterministic replay harness, no content-addressed state model, no proof-carrying commit path, no semantic/causal retrieval, no QTIC conformance, and no PSE/IL integration. Running `npm test` on the uploaded codebase executed Node's test runner but found zero tests.

Therefore, `nxhuman` is useful as a minimal IDE context bootstrapper. It is not structurally comparable to PSE. The correct response is not to clone it, but to extract its useful intention - IDE-agnostic AI context installation - and rebuild that intention as a PSE-native, post-symbolic layer.

1.2 Core Thesis

> `nxalien` is not a rules-file generator. It is a deterministic context exoskeleton that compiles project context into an executable Hypercube-HDAG, grounds an AI agent through path-invariant context artifacts, and emits only bridge-admissible PSE handoff candidates.

1.3 Relationship to Existing PSE

PSE already provides deterministic observation-to-crystal processing, Infinity Ledger integration, HDAG topology, Pfauenthron++ retrieval, QTIC certification, causal retrieval, knowledge clustering, epistemic health, lifecycle management, and epistemic agenda generation. `nxalien` should not duplicate those capabilities. It should sit above them as an agent-context layer that answers a narrower question:

> How should a coding agent be grounded, constrained, audited, and evolved inside a repository before any candidate output is allowed to approach PSE materialization?

2 Design Principles

P1. Bridge supremacy. `nxalien` MUST NOT directly construct `SemanticCrystal` or final emissions. It emits `NxAlienBundle`s and `NxAlienHandoffCandidate`s only. PSE-Bridge remains the sole commit path.

P2. Deterministic replay. Identical project state, policy, seed, and configuration MUST produce byte-identical canonical manifests and replay hashes.

P3. Rules are evidence-bound. A rule without scope, trigger, source evidence, severity, and validation path is advisory text, not a post-symbolic rule atom.

P4. Unknowns are first-class. Unresolved architectural choices MUST be explicit `UnknownSlots` with admissible domains, resolution evidence, and lifecycle state.

P5. Context is compiled, not dumped. The agent context MUST be generated from a canonical graph/cube representation, not handwritten prompt prose.

P6. Fail closed. If evidence is insufficient, replay is broken, governance fails, or path invariance cannot be checked, `nxalien` MUST produce `Hold` or `EvidenceOnly`, not a commit candidate.

P7. IDE agnostic, PSE native. Output MAY include `.rules`, `AGENTS.md`, `CLAUDE.md`, `.cursorrules`, or editor-specific adapters, but the authoritative artifact is the canonical `nxalien` manifest.

3 Positioning Against `nxhuman`

Dimension	`nxhuman`	`nxalien`
Primary artifact	Static `.rules` file plus `.nxlogs`	Canonical Agent Context Hypercube, executable HDAG, materialized context bundle, replay manifest
Runtime model	None after file creation	Deterministic compile/replay/verify pipeline
Memory	Append-only text log, not interpreted by code	Rule atoms, unknown slots, decisions, evidence, and context deltas as content-addressed state
Graph substrate	None	HDAG over rules, decisions, unknowns, evidence, specialists, gates
Formal state	JavaScript object embedded in generator	Canonical Rust structs serialized via JCS-compatible deterministic representation
Agent grounding	Prompt instructions	Gate-checked, budgeted, role-aware context blocks
Retrieval	None	Delegates to PSE+IL and optionally requests Pfauenthron++/causal retrieval
Governance	Informal principles	Constitutional preflight, severity, rule predicates, bridge-admissibility gates
Replay	None	Byte-identical replay hash and manifest verification
PSE compatibility	None	Observation adapter plus handoff candidate contract

4 Formal Model

4.1 Project Substrate

Definition 4.1 (Project Substrate)**.** *A project substrate is a tuple*

$$\Pi = (F, M, C, T, H, P)$$

where:

- F *is the finite set of selected repository files or file summaries,*
- M *is metadata: package manager, language, framework, license, CI surface, commands,*
- C *is the declared constraint set: style, safety, quality, architecture, governance,*
- T *is the tool surface: tests, lint, build, format, run, validation commands,*
- H *is historical state: decisions, unresolved unknowns, prior context bundles,*
- P *is the policy object controlling budgets, gates, and output targets.*

4.2 Rule Atom

Definition 4.2 (Rule Atom)**.** *A rule atom is a canonical object*

$$r = (id, scope, trigger, prescription, severity, evidence, gate, decay, hash)$$

where $severity \in \{advisory, required, blocking\}$ *and* $hash = H_{256}(JCS(r \setminus hash))$*.*

Requirement 4.1 (R-RULE-001)**.** *Every materialized rule MUST be derived from at least one source: default policy, repository evidence, user directive, previous accepted decision, or PSE/IL retrieval result. Source-less rules MUST be downgraded to advisory notes.*

4.3 Unknown Slot

Definition 4.3 (Unknown Slot)**.** *An unknown slot is a typed unresolved decision:*

$$u = (name, domain, status, candidates, evidence, resolution, confidence, expires).$$

The status is one of $\{unknown, proposed, resolved, stale, superseded\}$*.*

Unknown slots generalize `nxhuman`'s informal UNKNOWN markers. In `nxalien`, resolving an unknown is a state transition that must be logged, hashed, and optionally submitted as a PSE observation.

4.4 Agent Context Hypercube

Definition 4.4 (Agent Context Hypercube)**.** *An Agent Context Hypercube is*

$$Q_A = (C^n, G_A, \iota, \Xi_A, \Gamma_A, \Theta_A)$$

where:

- C^n *is an* n*-dimensional normalized context cube,*
- $G_A = (V, E, T, \Phi)$ *is an embedded HDAG over rule atoms, unknown slots, evidence, tools, specialists, and gates,*
- $\iota : G_A \to C^n$ *embeds nodes into cube coordinates,*
- Ξ_A *is the deterministic context compilation operator,*
- Γ_A *is the fail-closed nxalien gate family,*
- Θ_A *is the handoff policy mapping compiled artifacts to PSE bridge candidates.*

4.5 Recommended Coordinate Semantics

For a PSE-native implementation use C^8 internally, with a canonical projection to the PSE/HDAG 5D tensor.

Axis	Name	Meaning
ψ	Evidence potential	Strength of direct repository/user/PSE evidence
ρ	Rule density	Constraint concentration and rule specificity
ω	Temporal phase	Recency, lifecycle phase, or unknown-resolution phase
χ	Connectivity	Coupling to files, tools, tests, decisions, or agents
η	Causality	Directional dependency or decision lineage ordering
γ	Governance	Constitutional severity and policy burden
υ	Uncertainty	Lack of resolution, conflicting evidence, or stale state
λ	Utility	Expected value for next agent action

Projection into PSE/HDAG 5D space SHOULD use:

$$\pi_5(\psi, \rho, \omega, \chi, \eta, \gamma, \upsilon, \lambda) = (\psi, \rho, \omega, \chi, \eta')$$

with

$$\eta' = \text{clip}(0, 1, 0.50\eta + 0.25\gamma + 0.25(1 - \upsilon)).$$

5 HDAG Construction

5.1 Node Classes

Node Type	Description
`ProjectNode`	Canonical project metadata, language, package manager, test/build surface
`RuleNode`	Evidence-bound rule atom
`UnknownNode`	Typed unresolved decision slot
`EvidenceNode`	File, command result, user directive, PSE retrieval, or prior decision source
`SpecialistNode`	Agent role activation surface such as architect, security, backend, frontend, performance
`ToolNode`	Test, build, lint, format, audit, benchmark command
`GateNode`	Blocking/required/advisory rule predicate
`OutputNode`	Materialized context target such as `AGENTS.md`, `CLAUDE.md`, `.rules`, or PSE observation batch

5.2 Edge Causes

Edge Cause	Meaning
`DerivesFrom`	Rule or unknown derives from evidence
`Constrains`	Rule constrains file scope, tool, agent, or output
`Triggers`	Keyword/scope activates specialist behavior
`ValidatesBy`	Rule or output is validated by a tool command
`Resolves`	Decision resolves an unknown slot
`Supersedes`	New decision replaces a stale rule or prior decision

5.3 Semantic Coherence

For an edge (v_i, v_j) to be admissible, `nxalien` SHOULD evaluate:

$$R_A(v_i, v_j) = \psi_i \psi_j \rho_i \rho_j \cos(\omega_i - \omega_j)(1 - \max(v_i, v_j)).$$

The edge is inserted only if:

$$R_A(v_i, v_j) \geq \tau_A \quad \wedge \quad \eta_j \geq \eta_i - \epsilon_\eta.$$

This preserves directed causal drift while allowing small quantization tolerance.

Invariant 5.1 (I-ACYCLIC-001)**.** *The canonical* ***nxalien*** *HDAG MUST be acyclic. If a candidate edge would create a cycle, the implementation MUST either reject the edge or materialize it as a non-Hard* ***AdvisoryRelation*** *outside the executable HDAG.*

6 Compilation Semantics

6.1 Compilation Operator

Definition 6.1 (nxalien Compilation)**.** *The* ***nxalien*** *compilation operator*

$$\Xi_A : Q_A \to \mathcal{A}_A$$

materializes a finite artifact bundle:

$$\mathcal{A}_A = \{M_A, R_A, L_A, C_A, B_A, V_A\}$$

where:

- M_A *is* ***nxalien.manifest.json****,*
- R_A *is the generated rule/context text artifact family,*
- L_A *is the decision ledger append set,*
- C_A *is the agent context block for LLM injection,*
- B_A *is the PSE handoff candidate bundle,*
- V_A *is the replay/verification report.*

6.2 Reference Pipeline

```
ProjectRoot
  -> scan files + commands + existing context artifacts
  -> canonicalize ProjectSubstrate Pi
  -> extract RuleAtoms and UnknownSlots
  -> build Agent Context HDAG G_A
  -> embed G_A into C^8
  -> evaluate nxalien gates Gamma_A
  -> compile materialization layers
  -> emit manifest + context + logs + PSE handoff candidates
  -> replay hash + verify
```

6.3 Layered Materialization

Layer	Artifact	Purpose
L_0	`nxalien.manifest.json`	Canonical project/context state
L_1	`nxalien.rules.md`	Human-readable rules derived from rule atoms
L_2	`nxalien.context.json`	Machine-readable context for agents
L_3	`AGENTS.md`, `CLAUDE.md`, `.rules`	Optional IDE/agent adapters
L_4	`nxalien.replay.json`	Replay descriptor and hash chain
L_5	`NxAlienBundle`	Rust object for PSE bridge handoff

7 Gate Semantics

Definition 7.1 (nxalien Gate)**.** *The nxalien gate is the conjunction:*

$$G_A = G_{evidence} \wedge G_{scope} \wedge G_{replay} \wedge G_{canon} \wedge G_{delta} \wedge G_{budget} \wedge G_{governance} \wedge G_{bridge}.$$

Gate	Check
$G_{evidence}$	Every required/blocking rule has evidence or is downgraded/rejected
G_{scope}	Rules declare file, module, command, or project scope
G_{replay}	Re-running compilation over identical inputs yields identical replay hash
G_{canon}	Canonical serialization is stable and sorted
G_{delta}	Generated changes remain under configured diff/line budget unless explicitly elevated
G_{budget}	Context blocks fit token/character budget
$G_{governance}$	Blocking constitutional rules pass
G_{bridge}	No direct crystal/final emission is attempted; only handoff candidates are emitted

Gate outcome:

$$G_A(Q_A) \in \{reject, hold, evidence\_only, accept\}.$$

8 QTIC Compatibility

8.1 nxalien Context Attractor

A compiled `nxalien` context bundle becomes QTIC-compatible only when it is stable under replay, gate-passed, mirror-consistent between source repository and generated context, trace-ready, and path-invariant with respect to admissible compilation orders.

Definition 8.1 (nxalien Context Attractor)**.** *A context bundle $b^\star$ is an nxalien context attractor if:*

$$T_{\alpha,A}(b^\star) = b^\star$$

and:

$$Pass(G_A)(b^\star) \wedge MCI_A(b^\star) \geq \eta_A \wedge TraceReady(b^\star) \wedge Replay(b^\star) \wedge PathInv_A(b^\star).$$

8.2 Mirror Consistency

Let σ_S be source-derived context state and σ_M be materialized context state. Define:

$$MCI_A = 1 - d(\sigma_S, \sigma_M)$$

where d is a normalized distance over rule hashes, unknown slots, evidence references, tool commands, and generated adapter text.

Requirement 8.1 (R-QTIC-001)**.** *A generated IDE adapter file MUST NOT claim rules that are absent from the canonical* `nxalien` *manifest. Adapter text is a projection, not authority.*

9 PSE Integration Contract

9.1 Crate Layout

Recommended PSE workspace extension:

```
crates/
  pse-nxalien-types/ # canonical structs and schemas
  pse-nxalien-core/ # canonicalization, hashing, gates
  pse-nxalien-cube/ # Hypercube-HDAG embedding and compiler
  pse-nxalien-agent/ # context blocks, specialist activation, unknown resolution
  pse-nxalien-pse/ # ObservationAdapter + bridge handoff candidates
tools/
  nxalien-cli/ # init / inspect / compile / ground / resolve / replay / verify
adapters/
  pse-adapter-nxalien/ # optional high-level adapter if keeping adapter style
specs/
  NXALIEN_v1.0.0.tex
```

9.2 Rust Data Model

```
pub struct NxAlienRunDescriptor {
    pub schema_version: String,
    pub project_root_digest: String,
    pub policy_digest: String,
    pub seed: u64,
    pub started_at_utc: String,
}

pub struct RuleAtom {
    pub id: String,
    pub scope: ScopeRef,
    pub trigger: TriggerExpr,
    pub prescription: String,
    pub severity: Severity,
    pub evidence: Vec<EvidenceRef>,
    pub gate: Option<GateExpr>,
    pub decay: Option<DecayPolicy>,
    pub hash: String,
}

pub struct UnknownSlot {
    pub name: String,
    pub domain: Vec<String>,
    pub status: UnknownStatus,
```

```
    pub candidates: Vec<String>,
    pub evidence: Vec<EvidenceRef>,
    pub resolution: Option<String>,
    pub confidence: Fixed,
    pub expires: Option<u64>,
}

pub struct AgentContextCube {
    pub dimension: u8, // default: 8
    pub nodes: BTreeMap<String, NxAlienNode>,
    pub edges: Vec<NxAlienEdge>, // sorted canonical order
    pub embedding: BTreeMap<String, [Fixed; 8]>,
    pub policy: NxAlienPolicy,
}

pub struct NxAlienBundle {
    pub descriptor: NxAlienRunDescriptor,
    pub manifest_hash: String,
    pub context_hash: String,
    pub replay_hash: String,
    pub gate_report: NxAlienGateReport,
    pub artifacts: Vec<ArtifactDigest>,
    pub handoff_candidates: Vec<NxAlienHandoffCandidate>,
}

pub struct NxAlienHandoffCandidate {
    pub candidate_id: String,
    pub observation_bytes_digest: String,
    pub source_artifacts: Vec<ArtifactDigest>,
    pub intended_bridge: String, // e.g. "pse-bridge-v0.2"
    pub admissibility: BridgeAdmissibilityReport,
}
```

Invariant 9.1 (I-BRIDGE-001)**.** *The `pse-nxalien-*` crates MUST NOT import constructors that directly create `SemanticCrystal` or `FinalizedEmission`. The only permitted integration target is a PSE bridge or observation adapter interface.*

9.3 Observation Adapter

The adapter should convert `nxalien` artifacts into deterministic observation bytes:

```
impl ObservationAdapter for NxAlienObservationAdapter {
    fn observe(&self, bundle: &NxAlienBundle) -> Result<Vec<Vec<u8>>, AdapterError>
        {
        // 1. Serialize canonical manifest
        // 2. Serialize gate report
        // 3. Serialize context block
        // 4. Serialize replay descriptor
        // 5. Return sorted observation byte slices
    }
}
```

10 Agent Context Protocol

10.1 Context Block

```
[NXALIEN-CONTEXT version="1.0" replay="sha256:..."]
project: pse
mode: deterministic-agent-context
bridge_policy: candidate-only

[PRINCIPLES]
- evidence before assumptions
- minimal reversible changes
- tests before commit claims

[UNKNOWN_SLOTS]
- package_manager: resolved:cargo evidence:Cargo.lock
- task_runner: resolved:just evidence:justfile
- frontend_stack: unknown candidates:[none]

[RULES blocking]
- never emit SemanticCrystal directly; use PSE-Bridge only
- generated changes must include replay descriptor

[SPECIALISTS]
- architecture: triggers=["layer", "crate", "API", "adapter"]
- safety: triggers=["gate", "commit", "crystal", "governance"]

[AGENDA_HINTS]
- run cargo fmt --all -- --check
- run cargo clippy --workspace --all-targets --locked
- run cargo test --workspace --locked
[/NXALIEN-CONTEXT]
```

10.2 Specialist Activation

`nxalien` may preserve the practical `nxhuman` idea of specialist triggers, but each specialist must be converted into a typed `SpecialistNode` with trigger conditions, admissible actions, prohibited actions, and validation commands.

11 CLI Specification

Command	Function
`nxalien init`	Create `.nxalien/` directory, default policy, and first manifest
`nxalien inspect`	Print detected project substrate: files, commands, unknowns, evidence
`nxalien compile`	Build Agent Context Hypercube and materialize context artifacts
`nxalien ground`	Emit budgeted `[NXALIEN-CONTEXT]` for LLM system prompt injection
`nxalien resolve <slot>`	Resolve an unknown slot with evidence and append decision event
`nxalien handoff`	Emit PSE handoff candidates but do not commit
`nxalien replay`	Recompute hashes from manifest and artifacts
`nxalien verify`	Run gates, replay check, and optional project validation commands
`nxalien export -target claude`	Generate `CLAUDE.md` projection

`nxalien export -target agents`	Generate `AGENTS.md` projection

11.1 Minimal UX

```
# one-time setup
cargo run -p nxalien-cli -- init

# inspect detected context
cargo run -p nxalien-cli -- inspect

# compile canonical context and IDE projections
cargo run -p nxalien-cli -- compile --emit agents,claude,rules

# verify deterministic replay and gates
cargo run -p nxalien-cli -- verify

# produce PSE bridge candidates
cargo run -p nxalien-cli -- handoff --out .nxalien/handoff.json
```

12 Implementation Plan for Claude Code

12.1 Phase 0: Repository Integration

- Add workspace crates: `pse-nxalien-types`, `pse-nxalien-core`, `pse-nxalien-cube`, `pse-nxalien-agent`, `pse-nxalien-pse`.
- Add tool binary: `tools/nxalien-cli`.
- Add `specs/NXALIEN_v1.0.0.tex`.
- No existing PSE behavior may change in Phase 0.

Acceptance:

```
cargo fmt --all -- --check
cargo clippy --workspace --all-targets --locked
cargo test --workspace --locked
```

12.2 Phase 1: Canonical Types and Hashing

Implement canonical structs, deterministic sorting, JCS-compatible serialization, and replay hash generation.

Tests:

- identical manifests produce identical hashes,
- reordered input evidence produces identical canonical hash,
- changed rule severity changes manifest hash,
- missing evidence downgrades required rule or fails gate depending policy.

12.3 Phase 2: Project Scanner

Implement deterministic scanning for:

- package files: `Cargo.toml`, `package.json`, `pyproject.toml`, etc.,
- tool commands: tests, fmt, lint, build,
- existing context files: `AGENTS.md`, `CLAUDE.md`, `.rules`, `.cursorrules`,
- PSE metadata: README, CHANGELOG, specs, `Cargo.lock`.

12.4 Phase 3: Hypercube-HDAG Compiler

Build the graph, embed into C^8, evaluate coherence, topologically sort, and materialize layers. This is the central `nxalien` differentiator over `nxhuman`.

Tests:

- graph is acyclic,
- same project substrate yields same topological order,
- cycle attempts become advisory relations or fail closed,
- G_A rejects candidate bundles with direct crystal construction.

12.5 Phase 4: Context Projections

Generate `AGENTS.md`, `CLAUDE.md`, `.rules`, and `[NXALIEN-CONTEXT]` from the canonical manifest.

Invariant: generated text must be a projection. The manifest remains authoritative.

12.6 Phase 5: PSE Adapter

Implement observation adapter and handoff candidate generation. Do not modify PSE bridge internals unless strictly necessary.

Tests:

- handoff candidates are deterministic,
- adapter emits sorted byte slices,
- no `SemanticCrystal` is constructed inside `nxalien` crates,
- bridge integration test can ingest `nxalien` observations through the normal path.

12.7 Phase 6: Validation Demo

Create a demo that compares:

1. baseline coding agent without context,
2. static `.rules` style context,
3. `nxalien` compiled context,
4. `nxalien` + PSE/IL context.

Metrics:

- task completion,

- number of unsupported assumptions,
- number of test/lint failures,
- token budget consumed,
- replay match rate,
- invalid commit candidate rate.

13 Conformance Classes

Class	Name	Requirement
NXA-0	Parsed Context Candidate	Project substrate parsed; no determinism guarantee yet
NXA-1	Canonical Context Manifest	JCS-compatible canonical manifest and content hash
NXA-2	HDAG-Embedded Context	Rule/unknown/evidence graph embedded in C^8 and acyclic
NXA-3	Gate-Passed Context Bundle	G_A passes; materialized context fits budget and scope
NXA-4	Replayable Context Attractor	Replay hash verifies; mirror consistency threshold passes
NXA-5	PSE-Admissible Context Attractor	Bundle emits bridge-admissible handoff candidates and is QTIC-compatible

Only NXA-5 is eligible to influence PSE/IL materialization paths. Lower classes may be useful for IDE guidance but MUST NOT claim PSE epistemic status.

14 Security and Failure Policy

14.1 Failure Modes

Failure	Risk	Required Response
Context injection	Malicious file causes dangerous rules	Treat generated rules as evidence-bound; require scope and source
Rule drift	Adapter text diverges from manifest	Fail mirror consistency
Hidden commit path	Layer fabricates crystals	Compile-time deny import or test static source scan
Prompt bloat	Context exceeds useful budget	Budget gate holds or compresses
False authority	Advisory idea becomes blocking policy	Severity gate requires evidence
Stale unknown	Old decision no longer valid	Lifecycle decay to stale and reopen slot

14.2 Static Source Guard

Claude Code SHOULD implement a static test that scans `pse-nxalien-*` crates for forbidden constructors or imports. This is not a substitute for type-level boundaries, but it catches accidental bridge violations early.

15 Expected Value

`nxalien` would turn the surface promise of `nxhuman` into a real PSE layer:

- It keeps the IDE-agnostic context convenience.
- It replaces informal principles with evidence-bound rule atoms.
- It replaces plain text logs with content-addressed decision state.
- It replaces static prompt dumping with Hypercube-HDAG compilation.
- It makes unknowns, decisions, specialists, quality gates, and handoff policies replayable.
- It integrates cleanly with PSE by emitting candidates only.

This gives PSE a practical product-facing layer: an agent exoskeleton that can be installed into real repositories and used by coding agents, while preserving PSE's core invariants.

16 Definition of Done

The implementation is complete when:

1. `nxalien init`, `inspect`, `compile`, `ground`, `handoff`, `replay`, and `verify` work from the CLI.
2. Canonical manifests replay byte-identically across runs.
3. Generated IDE context artifacts are projections of the canonical manifest.
4. The Hypercube-HDAG graph is acyclic and deterministically sorted.
5. The gate report distinguishes reject, hold, evidence-only, and accept.
6. No `nxalien` crate constructs `SemanticCrystal` or `FinalizedEmission`.
7. PSE observation adapter integration works through the existing bridge path.
8. Workspace fmt, clippy, tests, and docs pass with locked dependencies.
9. At least one demo shows `nxalien` improving coding-agent assumption control versus a static rules file baseline.

17 Appendix A: Suggested README Description

```
nxalien is a PSE-native agent-context exoskeleton. It compiles repository
rules, unknown decisions, evidence, specialist triggers, validation commands,
and governance constraints into a deterministic Hypercube-HDAG context bundle.
The bundle materializes IDE-friendly projections such as AGENTS.md, CLAUDE.md,
and .rules, but the authoritative state is a content-addressed manifest with
replay proof, gate report, and PSE bridge handoff candidates.
```

18 Appendix B: Suggested GitHub Short Description

```
PSE-native post-symbolic agent-context layer: Hypercube-HDAG compiled rules,
unknowns, evidence, replay, gates, IDE projections, and bridge-safe handoff.
```

19 Appendix C: Source Alignment

This specification aligns with the following supplied materials:

- **Hypercube-HDAG Framework**: self-compiling n-dimensional containers, embedded HDAG execution, deterministic artifact generation, 5D coordinates, phase-gradient operators, and layered materialization.
- **HDAG v1.0**: 5D tensor graph semantics, semantic resonance, phase-gradient edges, and cognition by field formation rather than symbolic execution alone.
- **QTIC**: gate-, mirror-, trace-, replay-, and path-invariant information attractors, dual-fabric materialization, and conformance logic.
- **PSE README/CHANGELOG**: current PSE+IL substrate, HDAG, QTIC, causal retrieval, context compression, epistemic agenda, and bridge/fail-closed invariants.
- **nxhuman codebase**: minimal IDE-agnostic rules installer and append-only decision log seed pattern.